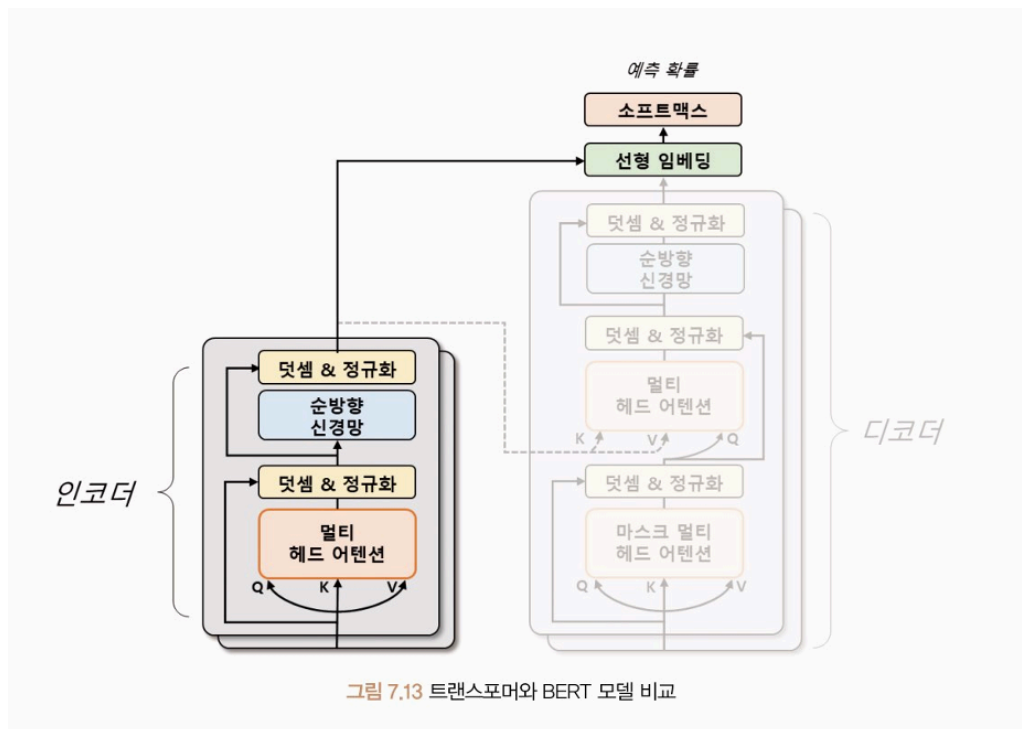


7장 트랜스포머(2)

BRET (Bidirectional Encoder Representations from Transformers)

2018년 구글에서 발표한 언어 모델로, 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델

- 양방향 (Bidirectional) 인코더 : 입력 시퀀스를 양쪽 방향에서 처리하여 이전과 이후의 단어를 모두 참조하면서 단어의 의미와 문맥을 파악하는 방식
→ 이전과 이후의 단어를 모두 참조해 문맥을 파악하므로 이전 단어만 고려하는 기존 모델들보다 더 정확하게 문맥을 파악하고 다양한 자연어 처리 작업에서 높은 성능을 보임.
- BRET 에서 사용되는 트랜스포머 모델



트랜스포머의 인코더만을 사용해 입력 문장 처리

사전 학습 방법

마스킹된 언어 모델링 (Masked Language Modeling, MLM)

입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식

⇒ 누락된 단어를 추론하는 능력을 갖게 되고, **문장 전체 의미 이해 능력 향상**

다음 문장 예측 (Next Sentence Prediction, NSP)

두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업.

ex) 'I'm learning PyTorch' 와 'Pytorch is a machine learning library' 라는 두 문장이 주어지면 BRET 은 이 두 문장이 연속적인 관계인지 아닌지 예측

⇒ 두 문장 간의 관계를 학습하고 문장 간의 의미적인 유사성 파악

특수 토큰 삽입

- [CLS] 토큰 : 입력 문장의 시작 부분에 추가되는 토큰으로 이 토큰을 이용해 문장 분류 작업을 위한 정보 제공
→ 모델이 입력 문장이 어떤 유형의 문장인지 미리 파악할 수 있게 됨
- [SEP] 토큰 : 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용되는 토큰
→ 입력 문장을 두 개의 독립적인 문장으로 인식하고 각각의 문장에 대한 정보를 정확하게 파악할 수 있게 됨.
- [MASK] 토큰 : 입력 문장 내에서 임의로 선택된 단어를 가리키는 특별한 토큰.
→ 마스킹된 실제 단어를 예측하는데 사용

```
#BRET 모델의 입력 임베딩의 구조  
[CLS] 문장1 [SEP] 문장2 [MASK] [SEP]
```

모델 실습

7.15 네이버 영화 리뷰 데이터 불러오기

```
import numpy as np  
import pandas as pd  
from Korpora import Korpora  
  
corpus = Korpora.load("nsmc")  
df = pd.DataFrame(corpus.test).sample(20000, random_state=42) #2000 개의  
train, valid, test = np.split(
```

```

    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))] #6
)

print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")

```

BERT 입력 텐서 생성

```

import torch
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

# 설정
epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"

```

```
tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased", #사전 학습
    do_lower_case=False #소문자로 변환하지 않게 함
)

# 데이터셋 및 데이터로더 생성
train_dataset = make_dataset(train, tokenizer, device) #텐서 데이터셋 반환
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])
```

- RandomSampler 클래스 : 데이터를 무작로 샘플링 → 학습에 적용
- SequentialSampler 클래스 : 데이터를 고정된 순서대로 반환 → 검증 및 평가 배치에 적용
- 결과

BERT 모델 선언

```

from torch import optim
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)

optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8) #활성화 함수

```

- BERT 문장 분류 모델 (BertForSequenceClassification) 클래스로 불러옴
- 패딩 기법을 사용하므로 설정 변경 X

모델 구조 확인

```

bert
├─ embeddings
│   ├── word_embeddings
│   ├── position_embeddings
│   └── token_type_embeddings
├─ LayerNorm
├─ dropout
├─ encoder
│   ├── layer
│   │   ├── 0
│   │   └── 1
│   └── ...
│       ├── 9
│       ├── 10
│       └── 11
├─ pooler
│   ├── dense
│   └── activation
├─ dropout
└─ classifier

```

- **입력 임베딩**
 - word_embedding (토큰 임베딩)
 - position_embedding (포지션 임베딩)
 - token_type_embedding (문장 구분 임베딩)
- **인코더 블록 12개 사용**

각각의 블록은 입력 시퀀스를 순차적으로 처리해 새로운 임베딩을 생성. → 입력 시퀀스의 다양한 관계 학습
- **풀러 (pooler)**

[CLS] 토큰 벡터를 한 번 더 비선형 변환을 수행하기 위해 선형 변환과 비선형 변환인 Tanh 함수를 적용 → Dropout(과적합 방지)
- **분류기 (Classifier)**

[CLS] 토큰 벡터를 활용해 결과 예측

학습 및 평가

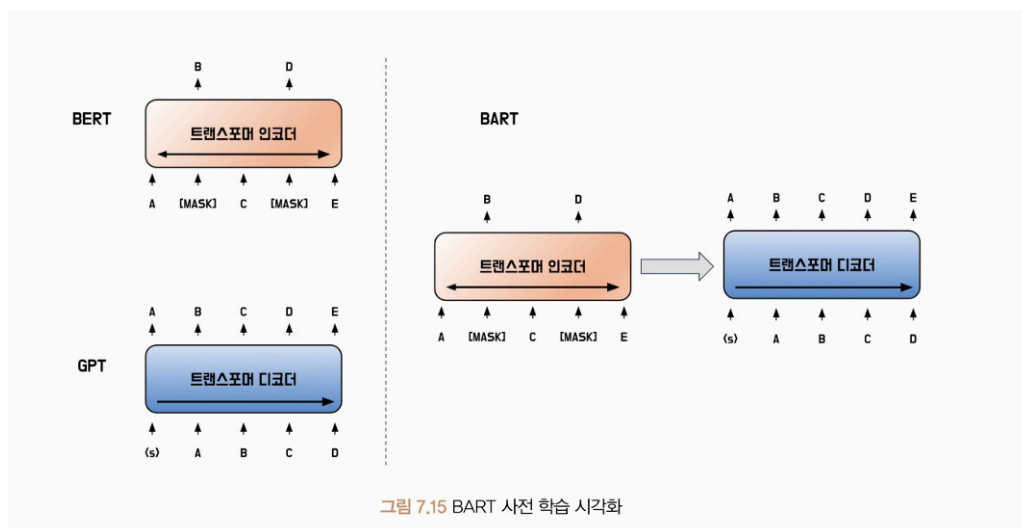
학습을 거듭할수록 학습 손실이 지속적으로 감소하며, 검증 손실 및 정확도가 개선된다.

```
Epoch 1: Train Loss: 0.4117 Val Loss: 0.4175 Val Accuracy: 0.8093
Saved the model weights
Epoch 2: Train Loss: 0.3277 Val Loss: 0.4329 Val Accuracy: 0.8067
Epoch 3: Train Loss: 0.2555 Val Loss: 0.4795 Val Accuracy: 0.8143
Epoch 4: Train Loss: 0.1922 Val Loss: 0.5138 Val Accuracy: 0.8103
Epoch 5: Train Loss: 0.1488 Val Loss: 0.5318 Val Accuracy: 0.8097
```

BART(Bidirectional Auto-Regressive Transformer)

2019년 메타의 FAIR 연구소에서 발표한 트랜스포머 기반의 모델

- BERT의 인코더와 GPT의 디코더를 결합한 시퀀스-시퀀스 (Sequence-to-Sequence) 구조
 - BERT의 인코더와 GPT의 디코더가 학습하는 방법을 일반화해 학습
- 노이즈 제거 오토인코더 (Denoising Autoencoder) 로 사전 학습
 - 입력 데이터에 잡음을 추가하고, 잡음이 없는 원본 데이터를 복원하도록 학습

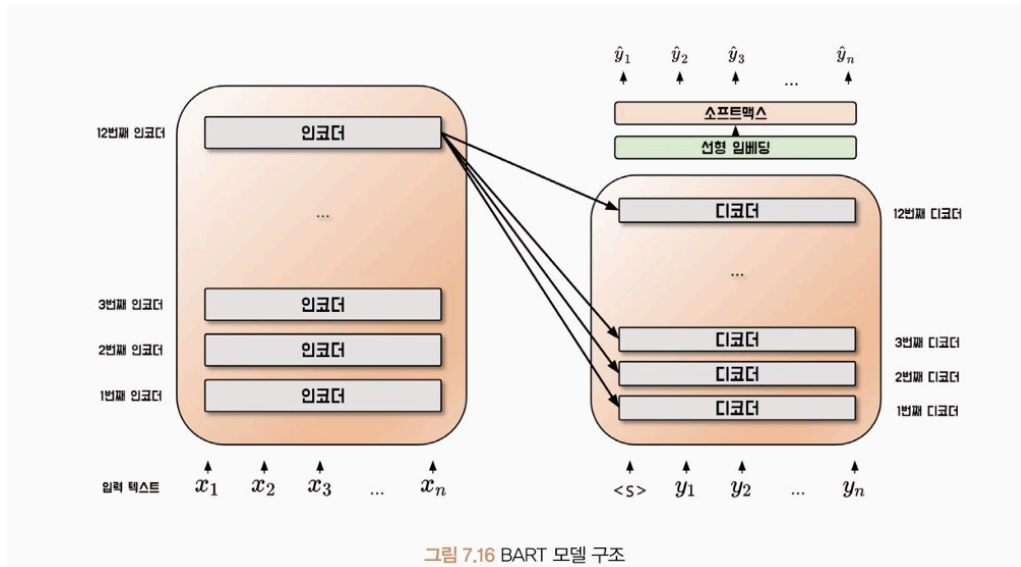


- 노이즈가 추가된 텍스트를 인코더에 입력하고 원본 텍스트를 디코더에 입력해 디코더가 원본 텍스트를 생성할 수 있게 학습하는 방식
 - BERT가 인코더 학습 시 텍스트에 마스킹을 추가하는 것과 같고 → GPT에서 순방향 정보만 인식할 수 있는 단점 개선

- GPT 가 디코더 학습 시 다음에 등장할 단어를 예측하는것과 같다 → 문장 생성 문
야에서 뛰어나지 않은 BERT 의 단점 개선

⇒ 문장 구조와 의미를 보존하면서 다양한 변형 학습 가능 !

- BERT 모델 구조



트랜스포머는 인코더의 모든 계층과 디코더의 모든 계층 사이의 어텐션 연산을 수행했다면
BART 는 **인코더의 마지막 계층과 디코더의 각 계층 사이에만 어텐션 연산 수행**

→ 정보 전달을 최적화하고 메모리 사용량을 줄일 수 있음

사전 학습 방법

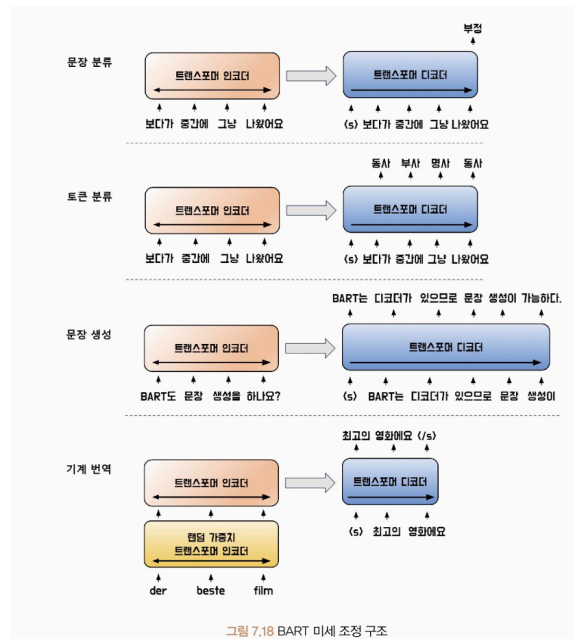
- BART 의 여러 노이즈 기법



1. 토큰 마스크 (Token Masking) : BERT 에서 사용한 MLM과 동일한 기법, 입력 문장의 일부 토큰을 마스크 토큰으로 치환
→ 문장 내에서 어떤 단어가 중요한 역할을 하는지 학습 가능
2. 토큰 삭제 (Token Deletion) : 입력 문장의 일부 토큰을 치환하는 것이 아니라 삭제하는 방법. 토큰 마스크와 다르게 어떤 위치의 토큰이 삭제되었는지도 맞춰야 함.
→ 입력 문장에서 불필요한 정보나 중요하지 않은 정보 자동 필터링. 학습과 예측 시간 감소, 일반화 성능 증가
3. 문장 순열 (Sentence Permutation) : 마침표를 기준으로 문장을 나눈 뒤 문장의 순서를 섞는 방법. 모델은 원래의 문장 순서를 맞춰야 함
→ 문장 내에서 단어들이 어떻게 연결되어 있는지 더 잘 파악하게 함. 문장이 다른 순서로 주어졌을 때도 잘 예측
4. 문서 회전 (Document Rotation) : 임의의 토큰으로 문서가 시작되도록 함. 문장의 순서는 유지되고, 모델은 원래 시작 토큰을 맞춰야 함.
→ 모델이 문서의 시작점을 인식할 수 있게 함.
5. 텍스트 채우기 (Text Filling) : 몇 개의 토큰을 하나의 구간(span) 으로 묶고 일부 구간을 마스크 토큰으로 대체. 모델은 연속된 마스크 토큰을 복구, 실제로는 마스크되지 않은 토큰도 구분해야 함
→ 모델이 누락된 단어를 예측하게 유도함으로써 입력 문장을 더 잘 이해할 수 있게 함.

미세 조정 방법

각 다운스트림 작업에 맞게 입력 문장을 구성해야 함



1. 문장 분류 :

- 입력 문장을 인코더와 디코더에 동일하게 입력
- 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력으로 사용

2. 토큰 분류 :

- 인코더와 디코더에 동일하게 입력
- 디코더의 각 시점별 마지막 은닉 상태를 토큰 분류기의 입력으로 사용
- 전체 입력과 각 시점별 은닉 상태의 어텐션 연산 수행

3. 문장 생성

- 추상적 질의응답 (Abstractive Question Answering)
- 문장 요약 (Summarization)

4. 기계 번역

- 사전 학습된 인코더에 기계 번역을 위한 인코더를 추가해 작업
- 새로 추가된 트랜스포머 인코더의 가중치와 위치 임베딩, 첫 번째 인코더 층의 셀프 어텐션 입력 행렬 가중치만 학습
 - 모든 신경망의 가중치를 작은 반복으로 학습

모델 실습

뉴스 요약 데이터셋 불러오기

```
import numpy as np
import pandas as pd

# Hugging Face Parquet 경로
splits = {
    'train': 'data/train-00000-of-00001-ebc48879f34571f6.parquet',
    'test': 'data/test-00000-of-00001-6227bd8eb10a9b50.parquet'
}

# test split에서 읽기
df = pd.read_parquet("hf://datasets/argilla/news-summary/" + splits["test"])

df["prediction"] = df["prediction"].apply(
    lambda x: eval(x)[0]["text"] if isinstance(x, str) and x.startswith("{") else x
)

# 2. 5000개 샘플링
df = df.sample(5000, random_state=42)

# 3. 다시 리스트 형태에서 텍스트만 꺼내기
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])

# 4. Train / Validation / Test 분할
train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6 * len(df)), int(0.8 * len(df))]
)

# 5. 출력
print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

- 책에 나온데로 하면 오류가 나서 허깅페이스 페이지에 가서 코드를 가져왔습니다!
- 자료형 문제

```
df["prediction"] = df["prediction"].apply(
    lambda x: eval(x)[0]["text"] if isinstance(x, str) and x.startswith("[{" )
)
```

⇒ 나중에 자료형 문

BART 입력 텐서 생성

```
import torch
from transformers import BartTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="max_length",
        max_length=512,
        truncation=True,
        return_tensors="pt"
    )

    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)

    labels = []
    for target in data.prediction:
        labels.append(tokenizer.encode(target, return_tensors="pt").squeeze())

    labels = pad_sequence(labels, batch_first=True, padding_value=-100).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
```

```

dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
return dataloader

# 하이퍼파라미터 설정
epochs = 3
batch_size = 8
device = "cuda" if torch.cuda.is_available() else "cpu"

tokenizer = BartTokenizer.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base" #사전 학습된 모델. B
)

# 데이터셋 및 데이터로더 생성
train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

- 패딩 값 -100 : 교차 엔트로피와 같은 손실 함수에서 패딩된 토큰을 무시하게 하기 위해 사용
- 데이터로더는 본문 텍스트의 정수 인코딩 값, 어텐션 마스크, 요약 문장의 정수 인코딩 값 반환

```

(
    tensor([ 0, 495, 1889, ..., 1, 1, 1], device='cuda:0'),
    tensor([1, 1, 1, ..., 0, 0, 0], device='cuda:0'),
    tensor([ 0, 35891, 161, 56, 5616, 10405, 19, 140, 23, 5490,
            3564, 2, -100, -100, -100, -100, -100, -100, -100, -100,
            -100, -100, -100, -100, -100, -100, -100, -100, -100, -100], device='cuda:0')
)

```

BART 모델 선언 & 구조

```
from transformers import BartForConditionalGeneration
from torch import optim

# 디바이스 설정 (GPU 사용 가능하면 사용)
device = "cuda" if torch.cuda.is_available() else "cpu"

# BART 모델 불러오기
model = BartForConditionalGeneration.from_pretrained("facebook/bart-base")

# 옵티마이저 설정
optimizer = optim.AdamW(model.parameters(), lr=5e-5, eps=1e-8) #조건부 함
```

- BART 조건 생성 (BartConditionalGeneration) 클래스 : BART 모델 중 하나로 조건부 생성 작업에 특화된 모델

```
model
├─ shared
├─ encoder
│  └─ embed_tokens
│  └─ embed_positions
│  └─ layers
│     └─ 0
│     └─ 1
│     └─ 2
```

```
│     └─ 3
│     └─ 4
│     └─ 5
└─ layernorm_embedding
├─ decoder
│  └─ embed_tokens
│  └─ embed_positions
│  └─ layers
│     └─ 0
│     └─ 1
│     └─ 2
│     └─ 3
│     └─ 4
│     └─ 5
└─ layernorm_embedding
lm_head
```

- 인코더와 디코더가 동일한 임베딩 계층 (shared 계층) 사용
→ 인코더와 디코더의 연결 강화
- 6개의 인코더와 디코더 계층
- 마지막 연산 layernorm_embedding : 인코더와 디코더에서 각 토큰의 임베딩에 적용되는 계층 정규화로 임베딩 벡터의 마지막 차원에 대해 정규화를 수행
- 인코더의 마지막 계층 출력값 - 디코더의 모든 계층과 어텐션 연산
- 디코더의 마지막 계층 출력 : 출력 크기가 단어 사전의 크기인 완전 연결 계층 통과

⇒ 언어 모델 생성

학습 및 평가

- 루지 (ROUGE) 함수 사용

생성된 요약문과 정답 요약문이 얼마나 유사한지를 평가하기 위해 토큰의 N-gram 정밀도와 재현율을 이용해 평가

- 루지 점수 계산 방법

정답 문장 유니그램	[대한민국, 은, 16, 강, 예, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 예, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$
정답 문장 바이그램	[대한민국은, 은 16, 16강, 강예, 예 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강예, 예 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$

- ROUGE-L : 생성된 요약문과 정답 요약문 사이에서 가장 공통부분 수열 (Longest Common Subsequence, LCS) 기반의 통계 방식
- ROUGE-LSUM : ROUGE-L 의 변형으로 텍스트 내의 개행 문자를 문장 경계로 인식하고 각 문장 쌍에 대해 LCS를 계산한 후 , union-LCS 라는 값 계산
 - union-LCS : 각 문장 쌍의 LCS 를 합집합, 중복 제거 후 길이를 계산한 값
 → 텍스트 요약에서 정확성과 완전성을 모두 반영
- ROUGE-W : 가중치가 적용된 LCS 방법
 - 단어 간 유사도를 고려해 요약 문장이 입력 문장의 의미를 더욱 잘 전달하는지 평가
- 코드

```
import numpy as np
import torch
```

```

from torch import nn
import evaluate

# ROUGE 스코어 계산 함수
def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1) #모델이 예측한 요약의 토큰 인덱스를 담은 2차원 텐서
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id) #정답 요약을 위한 토큰
    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    rouge2 = rouge_score.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )
    return rouge2["rouge2"]

# 학습 함수
def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        input_ids = input_ids.to(device)
        attention_mask = attention_mask.to(device)
        labels = labels.to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        train_loss += loss.item()

    optimizer.zero_grad()
    loss.backward()

```

```

optimizer.step()

train_loss = train_loss / len(dataloader)
return train_loss

# 평가 함수
def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss = 0.0
        val_rouge = 0.0

    for input_ids, attention_mask, labels in dataloader:
        input_ids = input_ids.to(device)
        attention_mask = attention_mask.to(device)
        labels = labels.to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        logits = outputs.logits
        loss = outputs.loss

        logits = logits.detach().cpu().numpy()
        label_ids = labels.to("cpu").numpy()
        rouge = calc_rouge(logits, label_ids)

        val_loss += loss.item()
        val_rouge += rouge

    val_loss = val_loss / len(dataloader)
    val_rouge = val_rouge / len(dataloader)
    return val_loss, val_rouge

```



```
# ROUGE 평가기 초기화
```

```
rouge_score = evaluate.load("rouge", tokenizer=tokenizer)
```

- calc_rouge 함수 : 텍스트 요약 작업에서 모델이 예측한 요약문과 정답 요약문 사이 루지 점수 계산

- 학습 결과

```
Epoch 1: Train Loss: 2.1503 Val Loss: 1.8627 ROUGE-2: 0.2579
Saved the model weights
Epoch 2: Train Loss: 1.6108 Val Loss: 1.8878 ROUGE-2: 0.2598
Epoch 3: Train Loss: 1.2267 Val Loss: 1.9929 ROUGE-2: 0.2495
```

- 평가 결과

```
Test Loss : 1.8220
Test ROUGE-2 Score : 0.2613
```

매우 작은 크기의 데이터로 샘플링해 학습을 수행했음을 고려하면 중간 수준의 성능을 보인다고 할 수 있다.

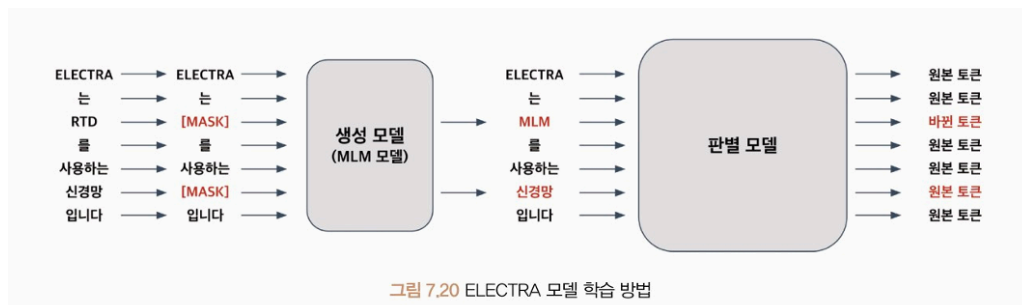
ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately)

2020년 구글에서 발표한 트랜스포머 기반의 모델

- 입력을 마스킹하는 대신 생성자 (Generator) 와 판별자 (Discriminator) 를 사용해 사전 학습 수행
- 생성적 적대 신경망 (GAN) 과 유사한 방법
 - 실제 데이터와 비슷하게 토큰을 생성해 다른 토큰으로 대체하고 판별 모델 생성 모델이 만든 데이터와 실제 데이터를 입력받아 어떤 데이터가 실제 데이터인지, 아니면 생성된 데이터인지 구분
- BERT 와 같은 모델과 비교했을 때 모델의 매개변수 수가 더 적음

사전 학습 방법

- 생성자, 판별자 모델 모두 트랜스포머 인코더 구조 따름
- 생성자 모델 - 입력 문장의 일부 토큰을 마스크 처리하고 마스크 처리된 토큰이 원래 어떤 구조였는지 예측하며 학습
 - BERT 의 마스킹된 언어 모델과 동일, 15% 를 마스크 처리
- 판별자 모델 - 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 인해 바뀐 토큰인지 맞히며 학습 (RTD (Replaced Token Detection) 방식)
 - 생성자 모델이 복원한 결과값을 입력받아 각 토큰이 원본 토큰인지 바뀐 토큰인지 판별



GAN 과 비슷하지만,

- 생성자 모델이 토큰을 정확히 예측한 경우 원본 토큰으로 인식하는 점
- GAN의 생성-판별 모델이 서로 적대적으로 (기능하게 어렵도록) 학습되는 것과 달리 ELECTRA의 생성 모델은 마스킹된 언어 모델을 통해 가려진 단어가 무엇인지 예측하도록 학습, 판별 모델은 각 토큰이 바뀌었는지 구분하도록 학습
- 두 모델이 같은 개수의 계층으로 이루어져 있다면 가중치를 공유할 수 있어 더 빠르게 학습 가능
 - ELECTRA 에서는 판별 모델의 크기를 1/2 나 1/4 로 줄이고 임베딩 계층의 가중치만 공유
 - 판별 모델이 학습하기 적합한 생성 모델을 유지하면서도 가중치를 공유하여 효율적으로 학습 가능
- 사전 학습이 완료되면 생성 모델을 사용하지 않고 판별 모델만 사용해 다운스트림 작업 수행
- 다운스트림은 BERT 와 같은 과정으로 미세 조정

모델 실습

네이버 영화 리뷰 데이터셋 전처리

BERT 에서 코드에 토큰나이저만 수정

```
# ELECTRA 토큰나이저
tokenizer = ElectraTokenizer.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discrimina
    do_lower_case=False
)
```

```
(tensor([ 2, 6511, 14347, 4087, 4665, 4112, 2924, 4806, 16, 3809,
         4309, 4275, 16, 3201, 4376, 2891, 4139, 4212, 4007, 6557,
         4200, 5, 5, 3, 0, 0, 0, 0, 0, 0,
         ...,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0], device='cuda:0'),
tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        ...,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0], device='cuda:0'),
tensor(1, device='cuda:0'))
```

모델 선언

```
from torch import optim
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discrimina
    num_labels=2
).to(device)

optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- KoELECTRA 모델 (한국어 분류 용)
- ELEECTRA 모델 - 문장 분류 모델 (ElectraForSequenceClassification) 클래스
- BERT 모델과 비교하기 위해 동일한 최적화 함수 AdamW, 매개변수 사용

모델 구조

```

electra
├── embeddings
│   ├── word_embeddings
│   ├── position_embeddings
│   ├── token_type_embeddings
│   ├── LayerNorm
│   └── dropout
├── encoder
│   ├── layer
│   │   ├── 0
│   │   └── 1
│   └── ...
│       ├── 10
│       └── 11
└── classifier
    └── dense

```

```

└── dropout
└── out_proj

```

- 입력 임베딩
 - 워드 임베딩
 - 위치 임베딩
 - 문장 구분 임베딩
- 임베딩 결합 후 계층 정규화, 드롭아웃
- 12개의 인코더 계층
 - BERT 와 동일
- 분류기 계층 : [CLS] 토큰 벡터를 이용해 분류 수행

학습 & 평가

- 학습

KoELECTRA모델 학습 결과

```

Epoch 1: Train Loss: 0.4489 Val Loss: 0.3227 Val Accuracy 0.8715
Saved the model weights
Epoch 2: Train Loss: 0.2891 Val Loss: 0.2928 Val Accuracy 0.8808
Saved the model weights
Epoch 3: Train Loss: 0.2188 Val Loss: 0.3111 Val Accuracy 0.8802

```

- 평가

Test Loss : 0.3084

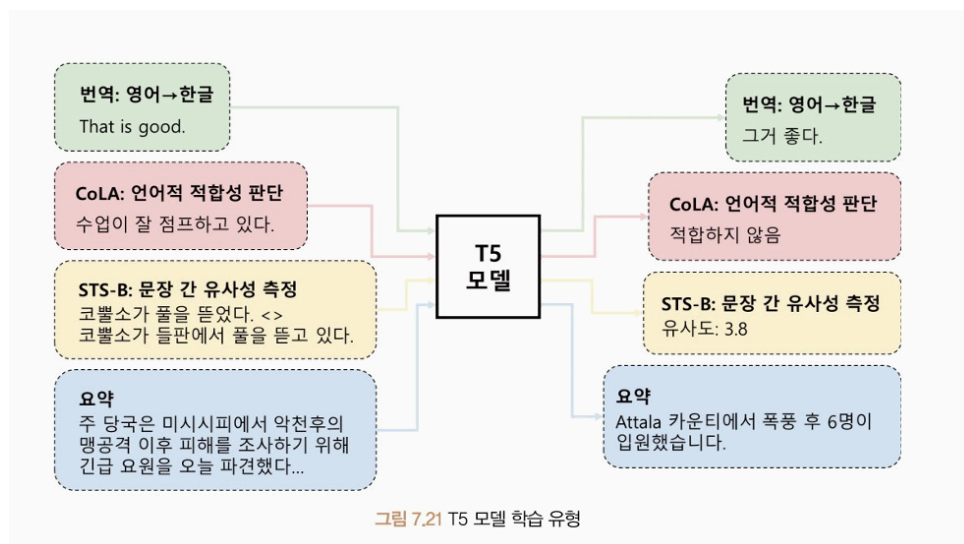
Test Accuracy : 0.8732

⇒ BERT 에 비해 높은 정확도를 가진다.

T5 (Text to Text Transfer Transformer)

2019년 구글에서 발표한 자연어 처리 분야의 딥러닝 모델

- 인코더-디코더 모델 구조를 바탕으로 GLUE, SuperGLUE, CNN/DM 등의 데이터세트에서 SOTA(State of the art) 를 달성했음.
- 입력과 출력을 모두 토큰 시퀀스로 처리하는 텍스트-텍스트 구조
→ 입력과 출력의 형태를 자유롭게 다룰 수 있으며, 모델 구조상 유연성과 확장성이 뛰어나 새로운 자연어 처리 작업에도 쉽게 적용 가능
- T5 모델 학습에 사용한 작업들



- CoLA 데이터세트 - 문법적으로 허용 가능한 문장과 그렇지 않은 문장 구분
- STS-B 데이터세트 - 문장 간 의미적 유사성 측정
- 학습 데이터세트 = 원본 문장과 대상 문장
 - C4 (Colossal Clean Crawled Corpus) 데이터세트를 활용해 사전 학습됨

- 비지도 학습으로 학습 - 입력 문장의 일부 구간을 마스킹해 입력 시퀀스를 처리, 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성
- 센티널 토큰 (Sentinel Token) : 문장마다 유일한 마스크 토큰을 의미
 - '<extra_id_0>' 과 같이 0부터 99개까지 100개의 기본값 사용



‘인코더-디코더 모델 구조’

입력 토큰의 센티널 토큰 : [<extra_id_0>,-,<extra_id_1>, 모델 구조]

출력 토큰의 센티널 토큰 : [인코더, <extra_id_0>, 디코더, <extra_id_1>]

- T5 는 이러한 마스킹 토큰을 예측하는 것을 목적으로 사전 학습
- 미세 조정 - 지도 학습 방식
 - 번역
 - 언어 수용성
 - 문장 유사도
 - 문서 요약
- 작업 토큰 : 문장이 인코딩되기 전에 추가
 - ‘translate English to German:’
 - ‘summerize’

모델 실습

7.26 뉴스 요약 데이터셋 불러오기

```
import numpy as np

import numpy as np
import pandas as pd

# Hugging Face Parquet 경로
```

```

splits = {
    'train': 'data/train-00000-of-00001-ebc48879f34571f6.parquet',
    'test': 'data/test-00000-of-00001-6227bd8eb10a9b50.parquet'
}

# test split에서 읽기
df = pd.read_parquet("hf://datasets/argilla/news-summary/" + splits["test"])

df = df[["text", "prediction"]].sample(5000, random_state=42)

# prediction 열이 ndarray이면 첫 번째 요소로 변환
df["prediction"] = df["prediction"].apply(lambda x: str(x[0]) if isinstance(x, (list, np.ndarray)) else x)

# T5 입력 prefix 붙이기
df["text"] = "summarize: " + df["text"]

# Train / Valid / Test 분할
train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6 * len(df)), int(0.8 * len(df))]
)

# 확인용 출력
print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")

```

- `text` = 뉴스 본문
- `prediction` = 요약된 뉴스
- 본문 앞에 `'summarize: '` 를 붙여 요약 작업이라는 정보를 모델 에 전달

7.27 뉴스 요약 데이터셋 전처리

7.19 와 유사

```

import torch
from transformers import T5Tokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler

def make_dataset(data, tokenizer, device):
    # 문자열 리스트로 변환
    text_list = data["text"].tolist()
    label_list = data["prediction"].tolist()

    # 문자열이 아니라 리스트 등으로 잘못 들어간 경우 방어 처리
    text_list = [t if isinstance(t, str) else str(t) for t in text_list]
    label_list = [l["text"] if isinstance(l, list) and isinstance(l[0], dict) else l for l in label_list]

    # prefix 붙이기
    text_list = ["summarize: " + t for t in text_list]

    source = tokenizer(
        text_list,
        padding="max_length",
        max_length=128,
        truncation=True,
        return_tensors="pt"
    )

    target = tokenizer(
        label_list,
        padding="max_length",
        max_length=128,
        truncation=True,
        return_tensors="pt"
    )

    source_ids = source["input_ids"].to(device)
    source_mask = source["attention_mask"].to(device)
    target_ids = target["input_ids"].to(device)
    target_mask = target["attention_mask"].to(device)

```



```
return TensorDataset(source_ids, source_mask, target_ids, target_mask)
```

- T5 토크나이저 클래스 (T5Tokenizer) : 사전 학습된 토크나이저 불러와 전처리
 - t5-small (T5 기본 버전)
- 패딩 방식 : ~~pad_to_max_length~~ 최대 길이 (max_length) 보다 짧으면 패딩 수행,
길명 문장을 자름
- input_ids (뉴스 본문과 요약된 뉴스의 토큰 인덱스), attention_mask (어텐션 마스크)
반환, 토큰화 결과 출력
- 토큰 인덱스 앞에 반복되는 21603, 10 은 본문 앞에 붙인 'summarize: ' 의미

```
[tensor([[21603, 10, 21603, ..., 3, 29155, 1],
        [21603, 10, 21603, ..., 809, 11, 1],
        [21603, 10, 21603, ..., 398, 888, 1],
        ...,
        [21603, 10, 21603, ..., 385, 315, 1],
        [21603, 10, 21603, ..., 30, 2875, 1],
        [21603, 10, 21603, ..., 0, 0, 0]], device='cuda:0'), tensor([[1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 1, 1, 1],
        ...,
        [1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 0, 0, 0]], device='cuda:0'), tensor([[ 3, 2, 31, ..., 0, 0, 0],
        [ 3, 2, 31, ..., 0, 0, 0],
        [ 3, 2, 31, ..., 0, 0, 0],
        ...,
        [ 3, 2, 31, ..., 0, 0, 0],
        [ 3, 2, 31, ..., 0, 0, 0],
        [ 3, 2, 31, ..., 0, 0, 0]], device='cuda:0'), tensor([[1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0],
        ...,
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0]], device='cuda:0')])
__summary__
:
```

결과

7.28 T5 모델 선언

```

from transformers import T5ForCon

model = T5ForConditionalGeneratio

# 전체 모델 구조 출력
print(model)

```

- shared 임베딩 함수 - 인코더와 디코더에 사용되는 토큰 임베딩으로 서로 공유되게 설정.
→ 인코더와 디코더의 각 계층에서 동일한 가중치가 사용되어 모델의 매개변수 수를 줄이고 일반화 성능 향상

T5 모델 구조 출력 결과

```

shared
encoder
  L embed_tokens
  L block
    | L 0
    | | L layer
    ...
    | L 5
    | | L layer
  L final_layer_norm
  L dropout
decoder
  L embed_tokens
  L block
    | L 0
    | | L layer
    ...
    | L 5
    | | L layer
  L final_layer_norm
  L dropout
lm_head

```

7.29 T5 모델 학습 및 평가

```

import numpy as np
from torch import nn

# 학습 함수
def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for source_ids, source_mask, target_ids, target_mask in dataloader:
        decoder_input_ids = target_ids[:, :-1].contiguous()

```

```

labels = target_ids[:, 1:].clone().detach()
labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100

outputs = model(
    input_ids=source_ids,
    attention_mask=source_mask,
    decoder_input_ids=decoder_input_ids,
    labels=labels,
)

loss = outputs.loss
train_loss += loss.item()

optimizer.zero_grad()
loss.backward()
optimizer.step()

return train_loss / len(dataloader)

# 평가 함수
def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss = 0.0

    for source_ids, source_mask, target_ids, target_mask in dataloader:
        decoder_input_ids = target_ids[:, :-1].contiguous()
        labels = target_ids[:, 1:].clone().detach()
        labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100

        outputs = model(
            input_ids=source_ids,
            attention_mask=source_mask,
            decoder_input_ids=decoder_input_ids,
            labels=labels,
        )

        loss = outputs.loss

```

```

        val_loss += loss.item()

    return val_loss / len(dataloader)

best_loss = 10000

for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss = evaluation(model, valid_dataloader)

    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "T5.pt")
        print("Saved the model weights")

```

- T5 모델 - 토큰 인덱스 (input_ids), 어텐션 마스크(attention_mask), 디코더 토큰 인덱스 (decoder_input_ids), 라벨(labels) 로 모델 학습
 1. decoder_input_ids : 디코더에 입력될 시퀀스를 인코딩한 텐서
마지막 토큰을 제외한 나머지 토큰 사용
 2. labels : decoder_input_ids 보다 다음 시점을 예측하도록 target_ids[:, 1:] 로 설정
 3. pad_token_id : -100 으로 설정해 손실값 계산 시 무시되도록 함
 - 출력 결과 학습/검증 손실이 점차 감소

출력 결과

```

Epoch 1: Train Loss: 4.3135 Val Loss: 3.3195
Saved the model weights
Epoch 2: Train Loss: 3.4098 Val Loss: 2.9162
Saved the model weights
Epoch 3: Train Loss: 3.1287 Val Loss: 2.7619
Saved the model weights

```

7.30 T5 생성 모델 테스트

```

model.eval()
with torch.no_grad():
    for source_ids, source_mask, target_ids, target_mask in test_dataloader:
        # 문장 생성
        generated_ids = model.generate(
            input_ids=source_ids,
            attention_mask=source_mask,
            max_length=128,
            num_beams=3,
            repetition_penalty=2.5,
            length_penalty=1.0,
            early_stopping=True,
        )

        # 생성 결과와 실제 타깃 비교
        for generated, target in zip(generated_ids, target_ids):
            pred = tokenizer.decode(
                generated,
                skip_special_tokens=True,
                clean_up_tokenization_spaces=True
            )
            actual = tokenizer.decode(
                target,
                skip_special_tokens=True,
                clean_up_tokenization_spaces=True
            )
            print("Generated Headline Text:", pred)
            print("Actual Headline Text :", actual)
            break # 배치 하나만 실행

```

- **생성 (generate) 메서드** : 입력 문장에 대한 요약문 (출력 시퀀스) 생성.
 - 토큰 인덱스 (input_ids) / 어텐션 마스크 (attention_mask) : 입력 문장의 인코딩과 마스크 정보 나타냄
 - 최대 길이 (max_length) : 생성될 요약문의 최대 길이
 - 빔 개수 (num_beams) : 빔 서치 (beam Search) 알고리즘의 빔 크기

⇒ 디코더 모델이 생성한 다수의 후보 단어 시퀀스 중 가장 높은 확률을 가진 시퀀스 선택해 출력

- 입력 시퀀스에 대한 출력 시퀀스 생성
- 반복 패널티 (repetition_penalty) : 중복 토큰 생성 제어
- 길이 패널티 (length_penalty) : 생성된 시퀀스 길이에 대한 보상. 값이 높을수록 긴 시퀀스 생성
- 조기 중단 (early stopping) : 최대 길이에 도달하기 전 EOS 토큰이 생성되는 경우 중단
- 디코딩(decode) 메서드
 - 생성된 출력 시퀀스를 디코딩
 - skip_special_tokens, clean_up_tokenization_spaces : 디코딩된 텍스트에서 특수 토큰과 불필요한 공백 제거